



Industrialisation de l'IA dans le cloud #1

YNOV Toulouse - M2 Data & IA

Intervenant Enzo DARDAILLON

Objectifs du cours

Industrialisation de l'IA dans la cloud

POC (Proof of Concept)

Passage du concept à la production

Scalabilité

Environnement

Performance

Sécurité

Déploiement

Monitoring

DataOps

CI/CD

Source	Date	% de projets en échec	Indicateurs
RAND	Août 2024	80%	Interview de 65 seniors Data Engineer + Scientist
Gartner	Juillet 2024	30% d'ici 2025	Prédiction basée sur de multiples facteurs
Gartner	Juin 2025	40% d'ici 2027	Prédiction basée sur de multiples facteurs
MIT	Juillet 2025	95%	300 déploiements analysé, 52 interviews techniques + 153 interviews dirigeants
S&P Global	2025	42%	Retours d'expérience de 1000 entreprises (NA + EU)

Quelques statistiques sur les projets IA mis en production



Pourquoi autant d'échecs ?

3 raisons principales

- IA ne répondant pas à un business
 - Aucun retour sur investissement
 - Coût trop élevé pour le gain potentiel
- Data Scientist “Top Career of the Decade”
 - Des Experts auto-proclamés brouillent les valeurs techniques
 - Profil orienté modèle mathématique et pas technique (POC ok mais production...)
- IA pour le marketing
 - On lance un produit IA → Ca fait vendre

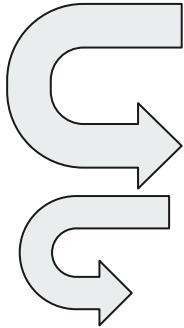
Comment industrialiser un projet Data ?



Un peu d'histoire...

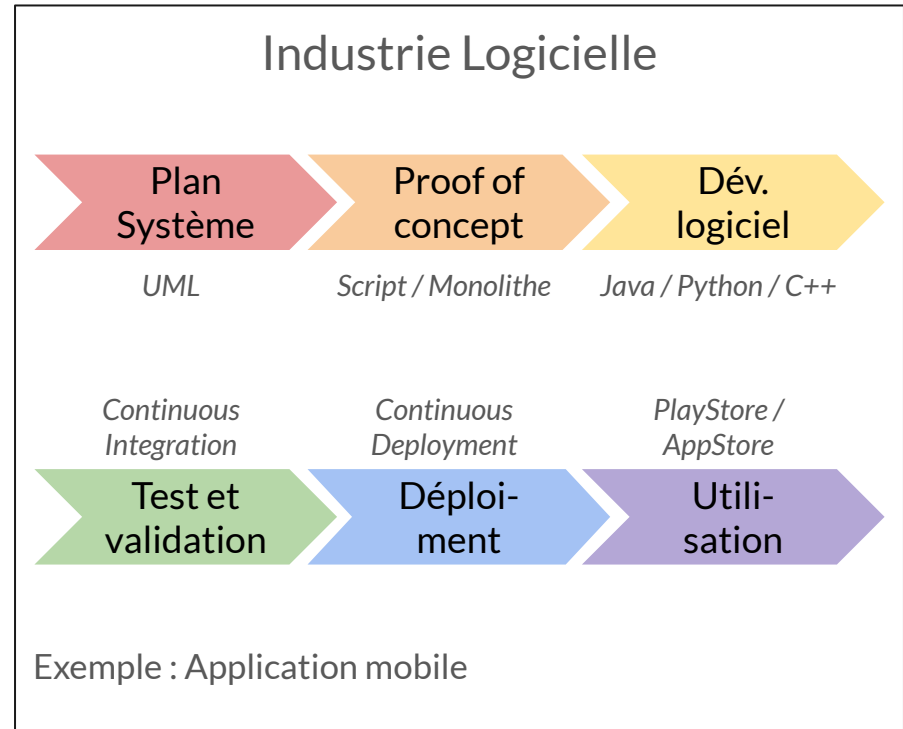
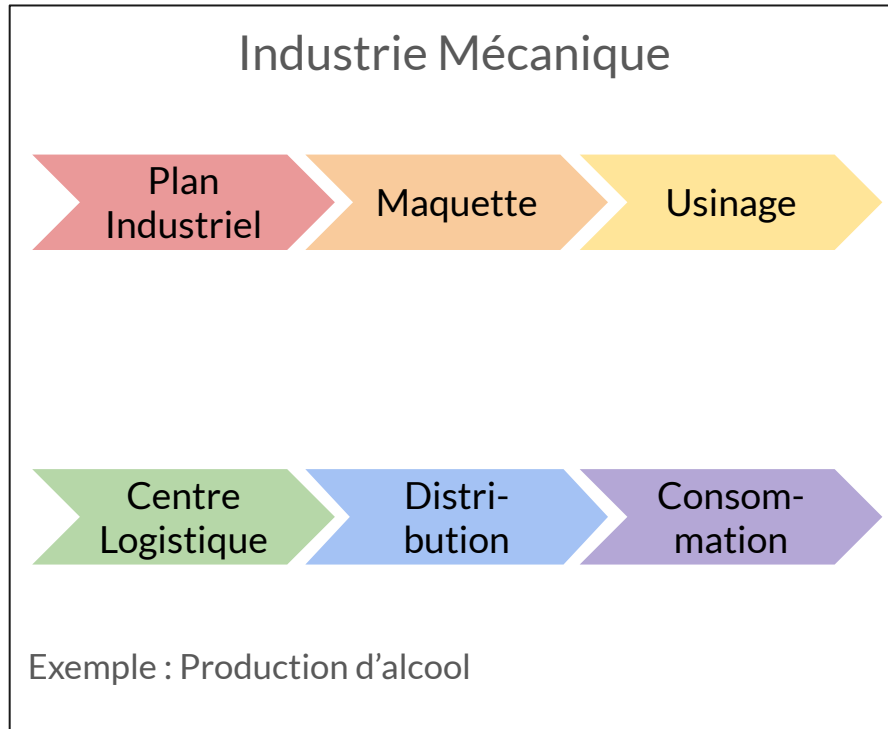
Début d'un
nouveau cycle

Début d'un
nouveau cycle ?

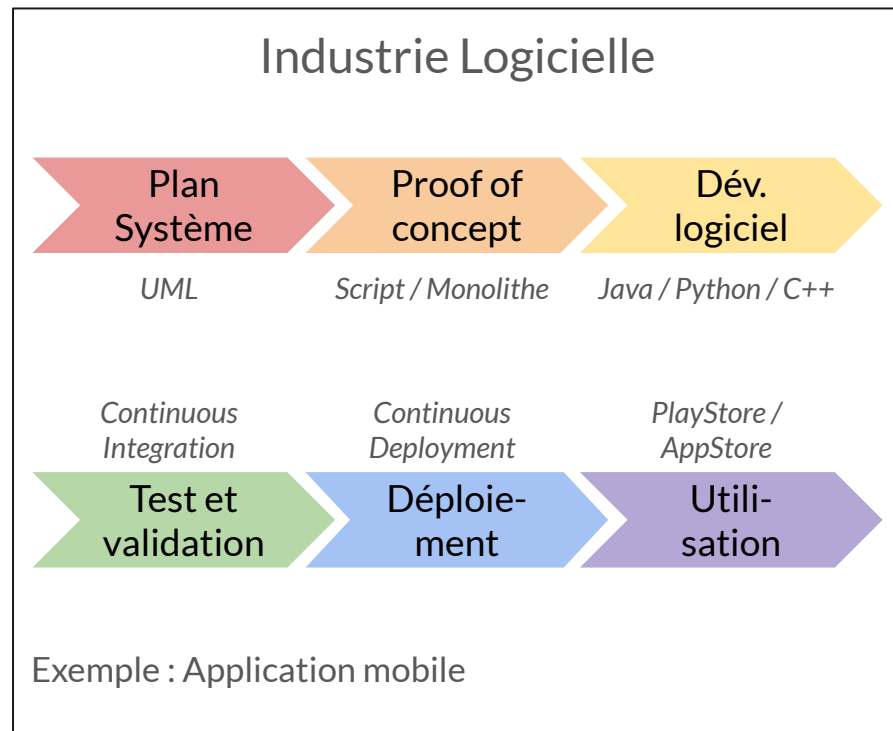
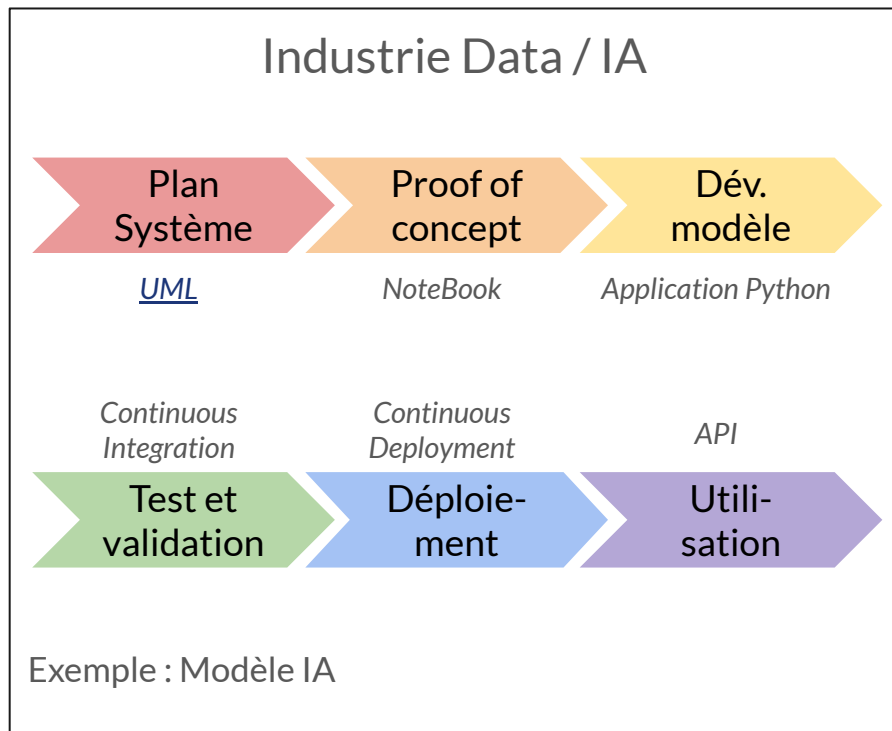


- 19ème siècle → Révolution industrielle
 - On arrête l'artisanat manuel et on remplace la force humaine ou animale par des machines
- 20ème siècle → Usinage à la chaîne
 - Les machines augmentent la cadence, on standardise, on met en place le travail à la chaîne
- Fin 20ème siècle → L'informatique facilite l'échange d'information
- Début 21ème siècle → Le développement d'application numérique se professionnalise
 - Création de méthodologie et théorisation du Génie Logiciel (Agile, Design Pattern, Scalabilité...)
- Aujourd'hui → L'IA facilite l'informatique
 - Les IA augmentent la cadence, aucune standardisation, pas de méthodologie propre au domaine

L'industrie Logicielle s'est basée sur l'industrie Mécanique



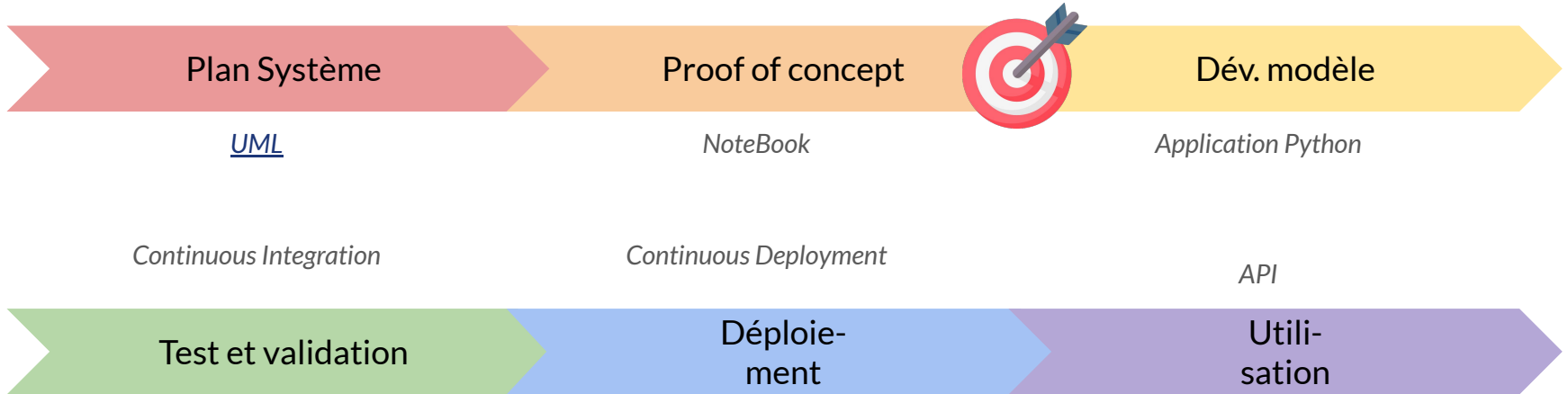
L'industrie Data/IA doit se baser sur l'industrie Logicielle



Objectif TP1



Industrie Data / IA



Exemple : Modèle IA

Comment passer du Concept à la Production ?

Notebook vs Application

**Quelles sont les différences
majeures entre Notebook et
application ?**

—

Petite parenthèse

“Environnement”, ça vous évoque quoi ?

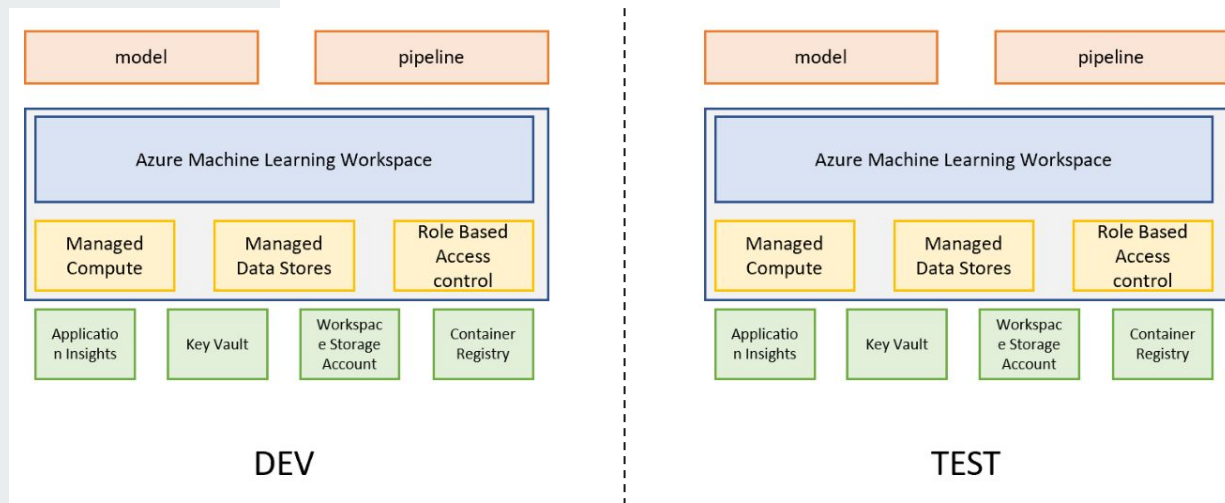
Séparation des environnements

dev → test → prod

Environnement ?

Définition et explication

- Ensemble de solutions techniques
 - Application
 - Base de données
 - Configuration
 - Service
 - Variable d'environnement
- Système isolé
 - Les entrées et sorties sont contrôlées et authentifiée



Source : [Microsoft Azure](#)

**Quelle utilité de séparer en
plusieurs environnements ?**

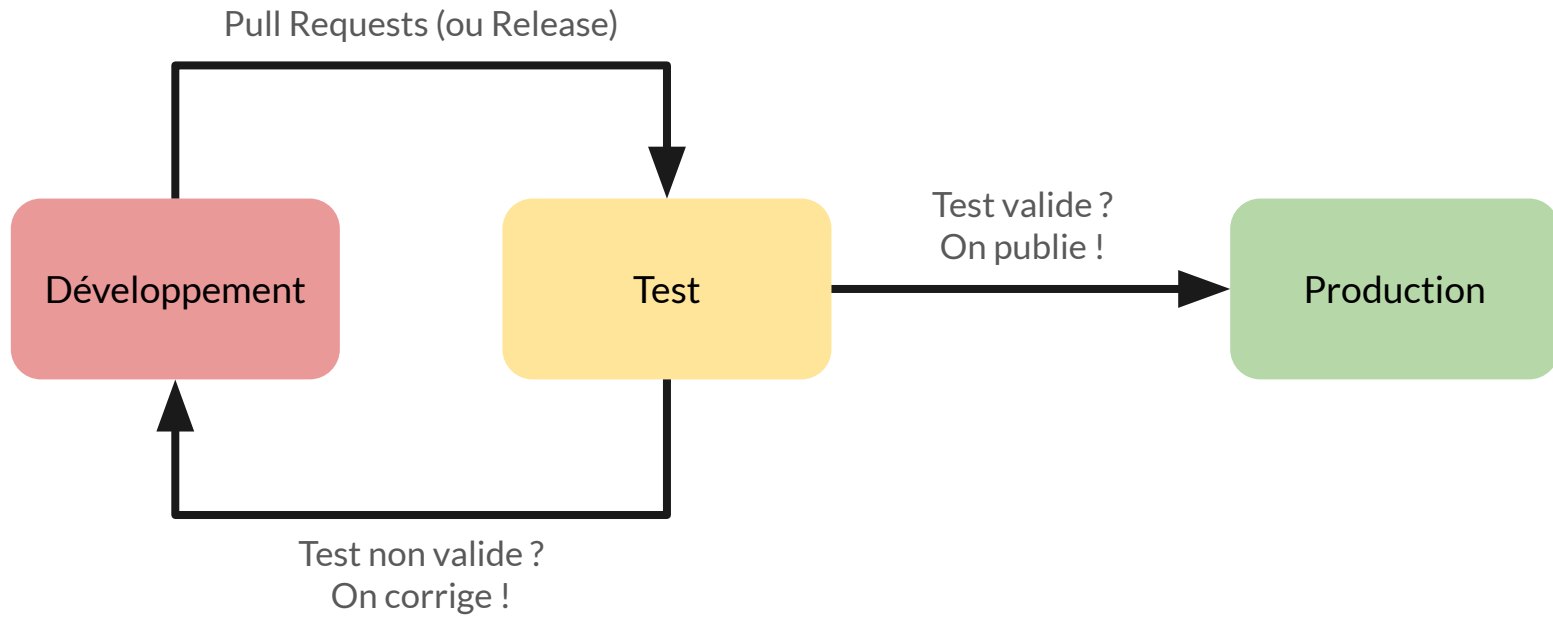
—



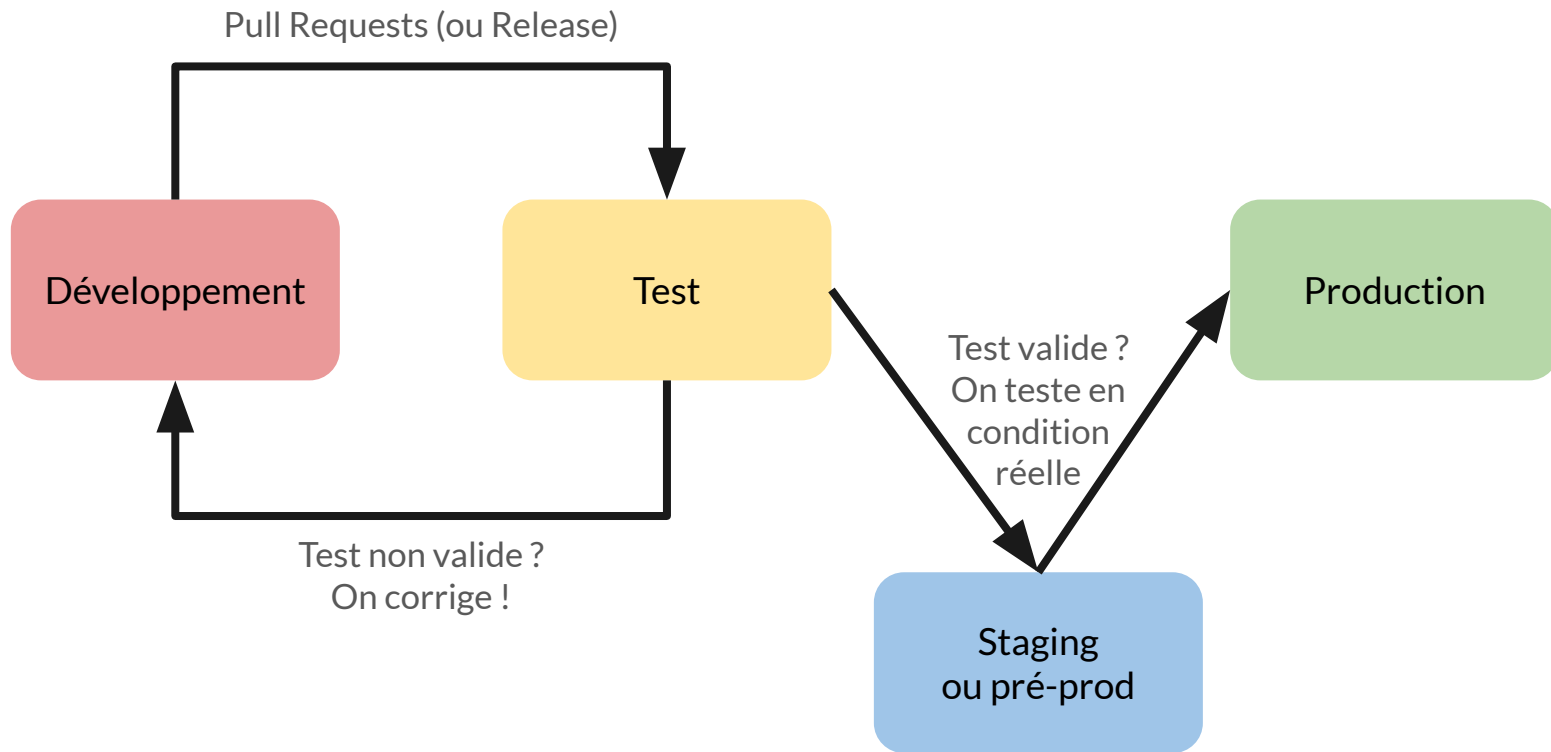
Séparation entre dev/prod

Pourquoi ?

- Pas d'impact sur l'utilisateur final avec des tests de développement
- Limitation des bugs sur l'application finale
- Risque de casser les données de la vraie base de données



Environnements dev → test → prod



[Alternative] Environnements dev → test → **staging** → prod



Différences

Environnements :

dev → test → prod

	Développement	Test	Production
Objectif	Expérimentation ou correctif	Validation et conformité	Satisfaire le besoin client
Utilisateur	Data Scientist ou développeur	Expert métier, Qualiticien	Client
Donnée	Générée aléatoirement, similaire prod	Échantillon contrôlé et anonyme de la prod	Réelle
Fiabilité	Faible, non voulue	Moyenne	Stricte

Zoom sur l'environnement de développement

Zone dédiée à l'ajout de fonctionnalité, la correction de bug et aux itérations rapides



Quelques règles :

- Aucun accès aux systèmes de production (Base de données, API...)
- Données anonymisées (car les nouvelles fonctionnalités n'ont pas passé les tests de confidentialité)
- Accès réservés aux équipes de développement
- On se rapproche au mieux de l'environnement de production

Un peu de concret :

- Budget faible : Exécution + Stockage local ou docker local (ordinateur du développeur)
- Vrai Budget : VPS ou cloud de développement

Zoom sur l'environnement de test

Zone dédiée au test des nouvelles fonctionnalités, corrections de bug



Vérifications effectuées :

- Test de Sécurité (confidentialité, accès non-autorisé...)
- Test de Non-régression (effet de bord...)
- Test d'intégrité des données
- Test de performance
- On se rapproche au plus de l'environnement de production

Un peu de concret :

- Budget faible : VPS ou GitHub Actions (si pas trop d'interfaces externes telles que UI, API, Database connector...)
- Vrai Budget : Identique à la production (avec moins de stockage et performance, suffisamment pour estimer les coûts et la consommation des ressources)

Zoom sur l'environnement de production

Zone accédée par les utilisateurs finaux



Quelques règles :

- Chaque accès (entrées et sorties) est contrôlée
- Les fonctionnalités sont testées
- Un plan de retour en arrière est prévu en cas de bug lors d'une montée de version
- Surveillance et monitoring de l'état de santé (metrics, traces, log)
- Validation manuelle (par un humain) avant de monter de version

Passage d'un environnement à l'autre

Ordinateur

GitHub

Senior

GitHub Actions

VPS ou cloud
minimal

Développement

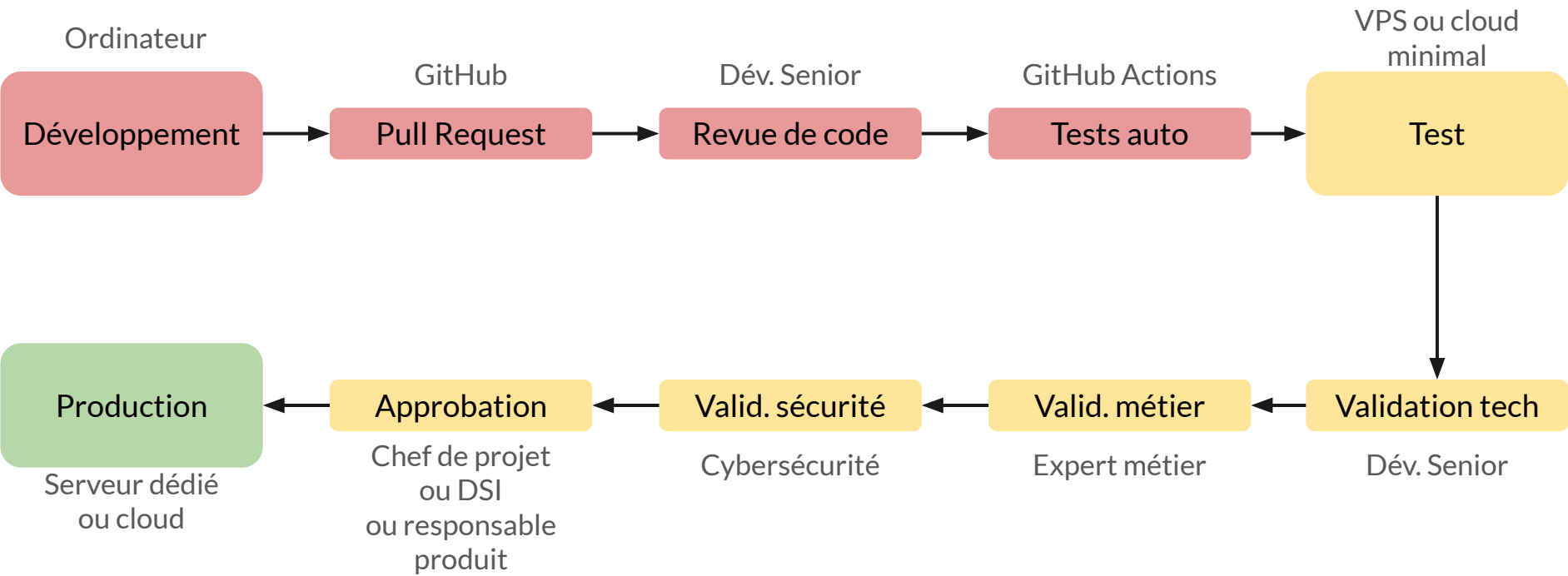
Pull Request

Revue de code

Tests auto

Test

Passage développement → test



Passage test → production

Fin de la parenthèse

Comment différencier une application
expérimentale et une application de production ?

Différence entre expérimentation et production



Expérimentation

- Données générées ou anonymisées
- On se concentre sur le COMMENT ?
- Pas de fiabilité
- Coût très faible

Production

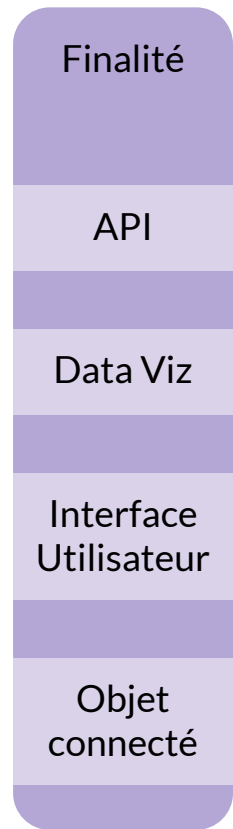
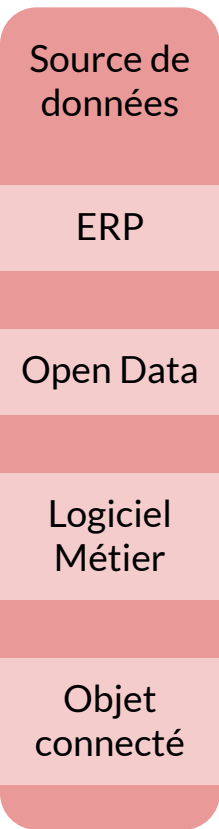
- Données sécurisées
- On se concentre sur le POURQUOI ?
- Fiable et sûr
- Coût plus élevé
- Surveillance continue

Conception d'une application Data en production

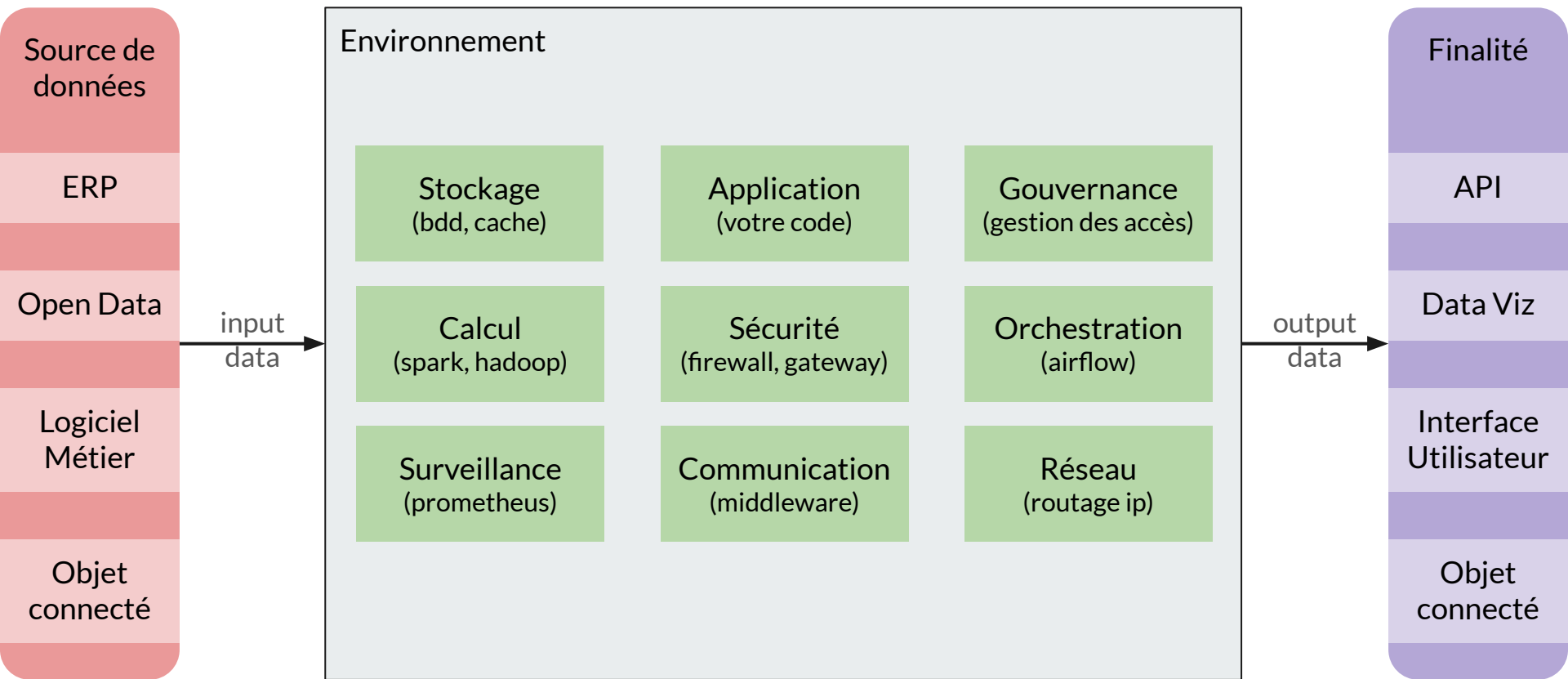
Avant de concevoir une application de production



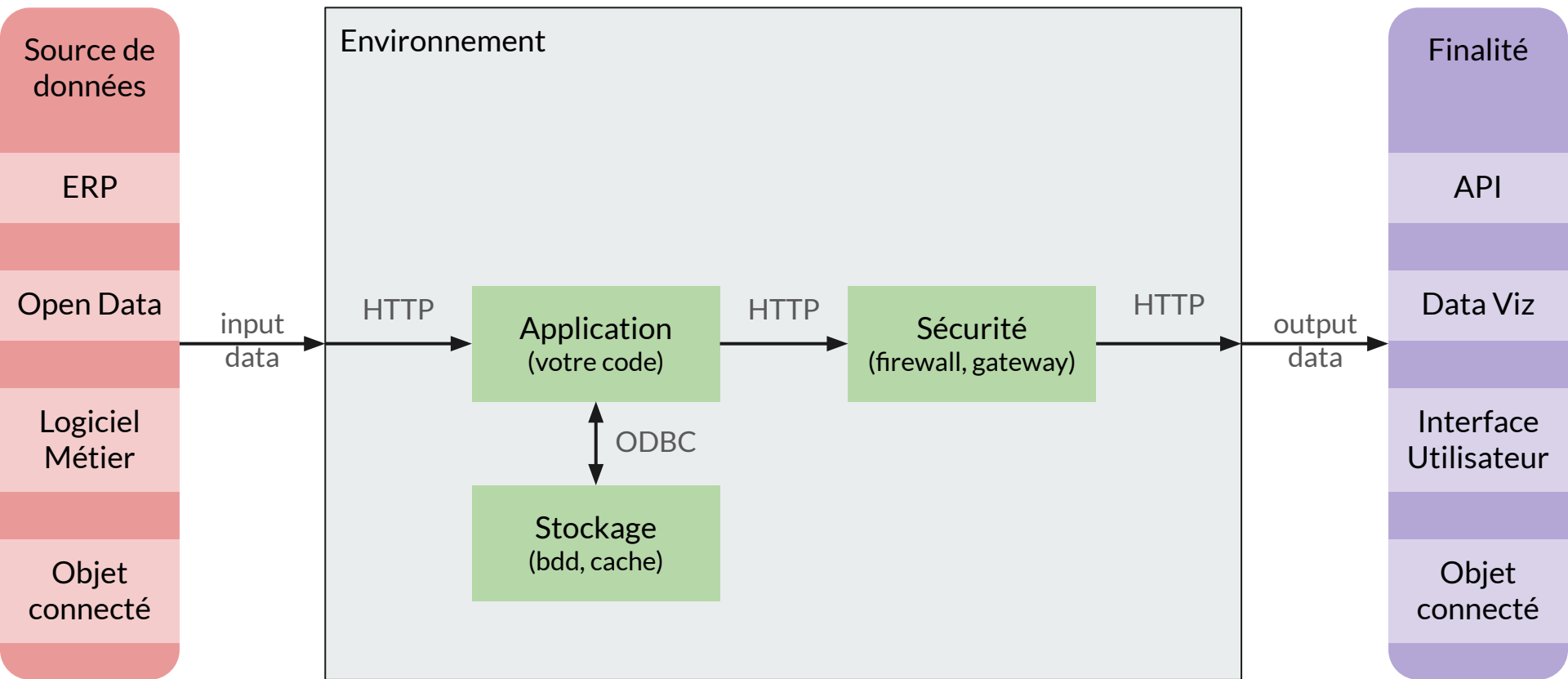
- 1) Comprendre le besoin final
- 2) Identifier les différentes parties prenantes du début à la fin de chaîne
 - a) Sources de données (début)
 - b) Data Viz ou Application IA (fin)
 - c) Capacité technique (intermédiaire)
 - i) Stockage
 - ii) Calcul
 - iii) Cloud



Les sources de données (début) et la finalité



Les sources de données (début) et la finalité + l'environnement



Exemple d'environnement très basique



L'architecture est dépendante du besoin final

C'est l'objectif du TP 1

Certains projets auront besoin de :

- Calcul distribué
- Performance temps-réel
- Confidentialité Secret Défense
- Auto-scaling
- Plusieurs types de base de données
- etc...

**Avant d'aller sur le projet,
Un vieux sénior m'a appris... ADR !**

ADR ?

Architecture Decision Records → Enregistrement des décisions d'architecture

ADR-0000 — Où stocker les artéfacts du modèle de Machine Learning

Status : Accepté

Date : 06/10/2026

Contexte

L'API doit pouvoir recharger un modèle au démarrage sans réentraîner à chaque redéploiement.

Le modèle fait quelques Mo, il est écrit peu souvent et lu très souvent.

Options considérées

Option	Avantage	Inconvénient
A. Répertoire local	Aucun service à installer	Pas de versionnage fiable
B. Système de fichiers réseau	Partage simple entre les deux machines	Accès concurrents pénibles
C. Model Registry ou équivalent	Versionnage natif	Service supplémentaire à maintenir

Décision

Option A car... Évolution probable vers l'option C dès 1M utilisateurs

Conséquence

Chaque version du modèle doit être identifiée explicitement

Le jour où 1M utilisateurs, cette ADR devra être marquée "Status: Superseded by ADR-000X" + nouvelle décision écrite.

**Avant d'aller sur le projet,
OpenAPI Spec (swagger) !**



OpenAPI Spec

Description / contrat d'une API
comportant endpoint + modèle

user Operations about user

POST `/user/createWithList` Creates list of users with given input array

GET `/user/{username}` Get user by user name

PUT `/user/{username}` Updated user

DELETE `/user/{username}` Delete user

GET `/user/login` Logs user into the system

GET `/user/logout` Logs out current logged in user session

POST `/user/createWithArray` Creates list of users with given input array

POST `/user` Create user

Models

ApiResponse >

**Avant d'aller sur le projet,
Rapide rappel Clean Code ?**



Clean Code (SOLID)

src/

└─ abstractions/ #class ModelRepository(ABC):

└─ infrastructure/

└─ dev/ #class FileModelRepository(ModelRepository):

└─ postgres/

En route vers le projet !

**Objectif : Transformer un Notebook en
Application de production**

**Sujet : Déterminer si une commande peut être
livrée dans la journée**



En route vers le projet !

Ce que je vous donne

- Un Notebook expérimental complet
 - Génération des données
 - Nettoyage des données
 - Sélection de variable
 - Entraînement de la Régression Logistique
 - Test du modèle
 - Évaluation du modèle
 - Prédiction
 - D'autres bouts de codes expérimentaux

Ce que vous devrez faire

- Globalement : Mettre en production cette application
- Étape 1 : Transformer un Notebook en application
- N'oubliez pas les principes de Clean Code
 - SOLID, DRY, YAGNI, Design Patterns...